HM Round: Contribution Backbone, Code to Defend It, and a Mock Interview

Prepared 2026-10-12 from: your resume, the A4S v2 High-Level Design (Jan 2026), `A4S_V2_COMPLETE_GUIDE.pdf`, `a4sdrill.md` / `drill2.md`, the GitLab master prep (table of contents and Salesforce chapters), and the LangGraph and LangSmith documentation.

Contents

HM Round: Contribution Backbone, Code to Defend It, and a Mock Interview
- Contents
- How to use this document
- Chapter 1: Your claim register
- Chapter 2: Pillar 1, planner and reasoner (V1 LangChain → V2 LangGraph)
- Chapter 3: Pillar 2, MCP server, Headless 360 and selective tool calling
- Chapter 4: Pillar 3, retrieval and grounding with connectors
- Chapter 5: "Flow of a production agent", infrastructure, and the other questions you were asked
- Chapter 6: Pillar 4, the resume, line by line
- Chapter 7: LangGraph fundamentals
- Chapter 8: LangSmith fundamentals
- Chapter 9: Mock hiring-manager interview
- Chapter 10: Study order and cheat sheet

How to use this document

1. **Chapter 1** is your claim register. Read it first; it tells you where each claim is strong and where an interviewer will push.
2. **Chapters 2–4** are your first three pillars (planner and reasoner; MCP and Headless 360; retrieval and grounding), each with the explanation, reference code and likely questions. **Chapter 5** covers the flow of a production agent, deployment and failure diagnosis. **Chapter 6** is the fourth pillar, your resume.
3. **Chapters 7–8** are the LangGraph and LangSmith fundamentals, so you can answer below the architecture level.
4. **Chapter 9** is the mock hiring-manager interview. Cover the answers and say yours aloud first.
5. **Chapter 10** is the study order and a numbers cheat sheet.

Evidence tags carry over from your drills: **[C]** verified in Core code, **[O]** verified in an off-core repo, **[S]** stated in Slack, **[HLD]** stated in the design doc, **[I]** inference, **[N]** searched for and not found. Code in this document is **reference code**: it shows the pattern so you can explain and reproduce it on a whiteboard. It is not a copy of the production source.

Chapter 1: Your claim register

1.1 Your four pillars, as you stated them

#	Your claim	What the files support	Where it is exposed
1	Wrote the planner and reasoner services. V1 was LangChain, V2 is LangGraph, supervisor → specialist	The reasoner repo has `atlas/` and `atlas_v2/` packages with different clients and timeouts (60 s vs 90 s) [O]. V2 graph has four nodes with invariants I-1..I-6 [C]. The HLD describes a Supervisor plus Knowledge, Reasoner and Executor specialists [HLD]. Planner type is `Atlas__ConcurrentMultiAgentOrchestration` [C]	See 1.2. The HLD names the graph runtime as an AI Cloud dependency
2	Wrote the MCP server logic, onboarded entities to Headless 360, selective tool calling	The Setup agent is exposed as an MCP server through a near-core proxy; session key is `(orgId, mcpSessionId, botId)`; proxy timeout 60 s with 2 retries; local latency sample p50 about 35 s, p95 about 98 s [drill2]. Tool retrieval uses top 5 at 0.7; documents carry a `tools:` list that sets `enabled_tools` [C]	Drill found one coarse tool wrapping the whole agent. The count of eight entities across 10+ teams was not found. The user-identity header was never read
3	Wrote retrieval and grounding, with connectors	Core `C360Retriever` → AI Gateway → Data Cloud hybrid search → rerank → Core filtering → LLM document selection [C/O]. Grounding check `GROUNDED` / `UNGROUNDED` in the planner trace [O]	Production chunk size, embedding model and fusion weights were not found. No calibrated confidence gate exists
4	Everything on the resume	Architecture, approvals, mTLS, graph design verified	Every number came back not found; the Slack Bolt app was not found; most of the infra bullet was not found. See chapter 6

1.2 The single biggest exposure: who owns the graph runtime

The HLD says v2 "leverages Agentforce 3.0 (Daisy + Agent Graph) for the runtime execution" and lists **"AI Cloud: Daisy, AgentGraph, File-based Agent"** under *Dependencies*. Its risks section says "we keep finding missing features and bugs within the new AgentGraph stack" and "Daisy++ will likely solve this." The A4S guide places the reasoner ("Atlas Daisy", a Python FastAPI service) and the planner (`bot-svc-llmagent`) on AI Cloud, with the agent definition YAML shipped in the Core jar.

An interviewer who has read that document will ask: "*The HLD treats Daisy and AgentGraph as an AI Cloud dependency. What exactly did you write?*"

Prepare to answer that question in one breath, with specifics. Two forms, to be used honestly according to what is true for you:

- **Full-ownership form (your stated claim):** "I wrote the planner and reasoner services. V1 was a LangChain topic-and-instruction loop; I rebuilt V2 as a LangGraph state graph with a supervisor and specialist nodes, and the YAML agent definition compiles into it."
- **Layered form (use if pressed on the platform boundary):** "I wrote the supervisor-to-specialist graph logic, the plan compilation and execution semantics, the planner relay and the confirmation behaviour. The platform team owns the shared Daisy runtime and AgentGraph primitives it builds on."

Whichever form is true, the rest of this document makes you defensible at the code level, because the follow-up is always "show me how."

1.3 Smaller things to settle in your own head before the round

- **Plan format.** Your `executionPlan` with `stages` and `execution_mode: parallel` is the HLD's **proposal** ("Feature Request: Parallel Action Execution"). The verified V2 runs a **todo list** (`upsert_todo_item`, `approve_todo_list`, `StartTodo → actions → UpdateTodoStatus`). Decide which is live and say so. Chapter 2.4 covers both.
- **Supervisor.** No `ConversationManager` service exists [N]. The role is split between the session worker (state in Redis) and the graph state. Say "the supervisor role."
- **mDLO.** The HLD describes metadata DLOs (mDLO) with an extraction status machine. The live corpus lands as a plain DLO from an S3 data stream [N for mDLO].
- **Proposals, not shipped:** parallel execution, "thinking out loud" streaming, generative UI (AXL) and Prism discovery.
- **The HLD is from January.** Do not quote its 10–20% latency gap as current.

Chapter 2: Pillar 1, planner and reasoner (V1 LangChain → V2 LangGraph)

2.1 The story in one minute

> V1 was a topic-and-instruction ReAct agent. Each topic had hand-written instructions and the agent picked from about 45 bespoke Java actions. That does not scale to the long tail of Setup tasks, it cannot ask a clarifying question before acting, and a plan built up front took 8–12 seconds, which reads as a dead agent. V2 moves control flow out of the model and into a graph. A supervisor routes between single-purpose specialists: Knowledge retrieves skill documents, the Reasoner turns the goal into a plan, the Executor runs the plan with no reasoning, and Discovery looks up org records. The model proposes; deterministic code decides what runs. The plan is shown to the user and approved before any write.

Why each choice, in the order an interviewer will ask:

Question	Answer
Why leave LangChain-style loops?	Control flow lived in the model. You cannot guarantee order, pause cleanly, or test a node in isolation. A graph makes "retrieve → plan → approve → execute" structural
Why a supervisor?	One place owns conversation state and delegation; control always returns to it after a specialist finishes, which keeps the flow resilient and resumable
Why single-purpose specialists?	Narrow tool sets reduce wrong-tool errors, and each node can be evaluated alone
Why is the Executor LLM-free?	What runs must equal what the user approved. If the executor re-reasons, that guarantee is gone
What did it cost?	Latency: the HLD reports the graph runtime 10–20% slower than the ReAct planner (Jan 2026), plus more hops and more timeouts

2.2 The two services

- **Reasoner** (`agent-svc-reasoner` , "Atlas Daisy"): runs the graph, calls the LLM at each step through the LLM Gateway, keeps graph state in Redis. About 5 LLM calls per request at P95 and 12 at P99; rate limit 200 requests per org per minute [O].
- **Planner** (`bot-svc-llmagent`): the seam between thinking and doing. It relays "execute action X" from the reasoner into Core, emits the deterministic confirmation message for tools with `requireUserConfirmation` , carries identity headers (`x-org-id` , `x-sfdc-user-id` , tenant id, app context) and publishes failures as `PlannerActionOutput` events [O/S].

The reasoner never talks to Core directly. That is deliberate: the component that thinks is not the component that touches the system of record.

2.3 Reference implementation: the supervisor-specialist graph

State carries everything the nodes share. Reducers matter: parallel steps write to `step_outputs` in the same super-step, so that key needs a merge function.

```
from operator import add
from typing import Annotated, Literal, TypedDict

from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, Send, interrupt


def merge_dicts(left: dict | None, right: dict | None) -> dict:
    return {**(left or {}), **(right or {})}


class AgentState(TypedDict, total=False):
    messages: Annotated[list, add]            # conversation
    user_ctx: dict                            # org, user, permissions (from the authenticated session)
    modality: Literal["information", "action"]
    skills: list[dict]                        # Knowledge output: selected NLD documents
    enabled_tools: list[str]                  # derived from skills' `tools:` lists
    plan: dict                                # ExecutionPlan JSON
    stage_idx: int
    step_outputs: Annotated[dict, merge_dicts]
    errors: Annotated[list, add]
    status: str                               # drives the supervisor's routing
```

The supervisor is a pure router over `status`. It is deterministic code; no LLM decides who goes next.

```
Route = Literal["triage", "knowledge", "reasoner", "approval", "executor", "respond"]

def supervisor(state: AgentState) -> Command[Route]:
    s = state.get("status")
    if s is None:                  return Command(goto="triage")
    if s == "triaged":             return Command(goto="knowledge")
    if s == "knowledge_done":
        return Command(goto="reasoner" if state["modality"] == "action" else "respond")
    if s == "planned":
        return Command(goto="approval" if plan_has_writes(state["plan"]) else "executor")
    if s == "approved":            return Command(goto="executor")
    if s == "stage_done":
        more = state["stage_idx"] < len(state["plan"]["stages"])
        return Command(goto="executor" if more else "respond")
    # declined, needs_clarification, failed, done
    return Command(goto="respond")
```

Specialists. Each does one job and writes one slice of state:

```
def triage(state: AgentState) -> dict:
    # Small, cheap LLM call that returns exactly one of two labels.
    modality = classify_intent(state["messages"][-1]["content"])  # "information" | "action"
    return {"modality": modality, "status": "triaged"}


def knowledge(state: AgentState) -> dict:
    result = retrieve(                        # chapter 4: parallel branches, rerank, permission filter
        query=state["messages"][-1]["content"],
        user=state["user_ctx"],
    )
    skills = select_documents(state["messages"], result.chunks)   # LLM picks; code validates the ids
    enabled = sorted({t for s in skills for t in s["tools"]})     # retrieval narrows the tool menu
    note = [] if not result.degraded else [f"retrieval degraded: {result.degraded}"]
    return {"skills": skills, "enabled_tools": enabled, "errors": note, "status": "knowledge_done"}


def reasoner(state: AgentState) -> dict:
    missing = find_missing_requirements(state["messages"], state["skills"])
    if missing:                                    # consultative loop: ask, do not guess
        return {"messages": [ask_user(missing)], "status": "needs_clarification"}
    plan = compile_plan(state["skills"], state["enabled_tools"])  # parallel RAG over actions, then one
LLM call
    validate_plan(plan, state["enabled_tools"])                   # raises on anything malformed
    return {"plan": plan, "stage_idx": 0, "status": "planned"}
```

The approval gate uses `interrupt()`. Resume re-runs the node from its first line, so everything before `interrupt()` must be side-effect free.

```
def approval(state: AgentState) -> Command[Literal["supervisor"]]:
    decision = interrupt({"type": "plan_approval", "plan": render_plan(state["plan"])})
    if decision.get("approved"):
        return Command(update={"status": "approved"}, goto="supervisor")
    return Command(update={"status": "declined"}, goto="supervisor")
```

The Executor has no LLM. A parallel stage fans out with `Send`; a serial stage runs its steps in order.

```
def executor(state: AgentState) -> Command:
    stage = state["plan"]["stages"][state["stage_idx"]]
    ctx = {"outputs": state.get("step_outputs", {}), "enabled": state["enabled_tools"],
           "user": state["user_ctx"]}

    if stage["execution_mode"] == "parallel" and len(stage["steps"]) > 1:
        return Command(goto=[Send("run_step", {"step": s, **ctx}) for s in stage["steps"]])

    outputs, errors = {}, []
    for step in stage["steps"]:                          # serial: each step may use the one before
        try:
            outputs[step["stepId"]] = execute_step(step, {**ctx["outputs"], **outputs},
                                                   ctx["enabled"], ctx["user"])
        except ToolError as e:
            errors.append({"step": step["stepId"], "error": str(e)})
            break                                        # fail fast on writes
    return Command(update={"step_outputs": outputs, "errors": errors, "stage_idx": state["stage_idx"] +
1,
                           "status": "failed" if errors else "stage_done"}, goto="supervisor")


def run_step(payload: dict) -> dict:                     # one parallel branch
    step = payload["step"]
    try:
        out = execute_step(step, payload["outputs"], payload["enabled"], payload["user"])
        return {"step_outputs": {step["stepId"]: out}}
    except ToolError as e:                               # reads degrade; the stage still completes
        return {"errors": [{"step": step["stepId"], "error": str(e)}]}


def advance(state: AgentState) -> dict:                  # runs once after all parallel branches finish
    return {"stage_idx": state["stage_idx"] + 1,
            "status": "failed" if state.get("errors") and plan_stage_has_writes(state) else
"stage_done"}
```

Wiring and compile. Every specialist returns control to the supervisor.

```
b = StateGraph(AgentState)
for name, fn in [("supervisor", supervisor), ("triage", triage), ("knowledge", knowledge),
                 ("reasoner", reasoner), ("approval", approval), ("executor", executor),
                 ("run_step", run_step), ("advance", advance), ("respond", respond)]:
    b.add_node(name, fn)

b.add_edge(START, "supervisor")
for specialist in ("triage", "knowledge", "reasoner"):
    b.add_edge(specialist, "supervisor")
b.add_edge("run_step", "advance")
b.add_edge("advance", "supervisor")
b.add_edge("respond", END)
# supervisor, approval and executor route with Command(goto=...)

graph = b.compile(checkpointer=checkpointer)   # durable store required for interrupt()

config = {"configurable": {"thread_id": session_id}}
graph.invoke({"messages": [user_msg], "user_ctx": ctx}, config)          # runs until the approval
interrupt
graph.invoke(Command(resume={"approved": True}), config)                  # next HTTP request resumes
```

What this maps to in the real system: the `status` routing is the supervisor role; `triage/knowledge/reasoner/executor` are the specialists; Discovery is another specialist node that writes org lookups into state; checkpoint state is Redis in the Setup agent [O] and Postgres in ResolveFlow (your own project).

2.4 The plan and how it executes

Two plan shapes exist in your files. Know both.

Shape A, the staged plan (HLD proposal for parallelism). Corrected version of your file (the typos in `finalplan.json` fixed):

```
{
  "executionPlan": {
    "stages": [
      { "stageId": "stage_1_find_user", "execution_mode": "serial",
        "steps": [ { "stepId": "step_1_find_user", "tool": "FindUser",
                     "inputs": { "name": "Ravi" } } ] },
      { "stageId": "stage_2_gather_assignments", "execution_mode": "parallel",
        "steps": [
          { "stepId": "step_2_find_perm_sets",   "tool": "FindPermissionSets",
            "inputs": { "userId": { "$ref": "step_1_find_user.userId" } } },
          { "stepId": "step_3_find_public_groups", "tool": "FindPublicGroups",
            "inputs": { "userId": { "$ref": "step_1_find_user.userId" } } },
          { "stepId": "step_4_find_queues",      "tool": "FindQueues",
            "inputs": { "userId": { "$ref": "step_1_find_user.userId" } } }
        ] }
    ]
  }
}
```

Fixes to your file: `"sertial"` → `"serial"`; remove the nested `steps` inside stage 1; `"tools"` → `"tool"`; stage 2 should list all three lookups.

Shape B, the todo list (verified in V2). The Assistant builds `todo` items, each backed by one NLD document; the turn ends and shows the plan; `approve_todo_list` sets `todo_approved`; single-task plans auto-approve. Execution is `StartTodo → actions → UpdateTodoStatus`, so any failure is attributable to one task.

How to explain Shape A in one breath: *Stage 1 resolves the name to a user id and must finish first. Stage 2 is three read-only lookups that depend only on that id, so they fan out and the stage takes as long as the slowest step. Data flows by `$ref`, resolved by code, never copied by the model. A stage with writes would be serial and would sit behind the approval gate.*

The two helpers an interviewer will ask you to write:

```
def resolve_refs(value, outputs: dict):
    """Replace {"$ref": "stepId.field"} with the earlier step's output. Code, not the LLM."""
    if isinstance(value, dict):
        if set(value) == {"$ref"}:
            step_id, _, field = value["$ref"].partition(".")
            try:
                return outputs[step_id][field]
            except KeyError:
                raise ToolError(f"unresolved reference {value['$ref']}")
        return {k: resolve_refs(v, outputs) for k, v in value.items()}
    if isinstance(value, list):
        return [resolve_refs(v, outputs) for v in value]
    return value


def validate_plan(plan: dict, enabled_tools: list[str]) -> None:
    seen: set[str] = set()                                  # step ids from EARLIER stages only
    for stage in plan["executionPlan"]["stages"]:
        if stage["execution_mode"] not in ("serial", "parallel"):
            raise PlanError("bad execution_mode")
        stage_ids = set()
        for step in stage["steps"]:
            if step["tool"] not in enabled_tools:
                raise PlanError(f"tool not enabled: {step['tool']}")
            if step["stepId"] in seen | stage_ids:
                raise PlanError("duplicate stepId")
            for ref in iter_refs(step["inputs"]):
                if ref.split(".")[0] not in seen:           # not same stage, not forward, not
unknown
                    raise PlanError(f"invalid reference {ref}")
            check_input_schema(step["tool"], step["inputs"])  # additionalProperties: false
            if stage["execution_mode"] == "parallel" and is_write(step["tool"]):
                raise PlanError("writes may not run in a parallel stage")
            stage_ids.add(step["stepId"])
        seen |= stage_ids
```

Be honest about validation: your drill found **no runtime validation of a generated plan beyond the approval itself** [N]. Present `validate_plan` as the pattern and the improvement, not as something already shipped.

2.5 `execute_step`: where authorization lives

```
def execute_step(step: dict, outputs: dict, enabled: list[str], user: dict):
    tool = step["tool"]
    if tool not in enabled:                                 # menu gate
        raise ToolError(f"tool not enabled: {tool}")
    args = resolve_refs(step["inputs"], outputs)
    check_input_schema(tool, args)                          # reject unknown / malformed fields
    return core_client.call(                                # planner -> Core; identity from the
session, never from args
        tool, args, user=user,
        timeout=60,                                         # no retry for writes: not idempotent yet
    )
```

The model produced `step`. Everything that decides whether it runs (menu gate, schema, permission check in Core, confirmation) is code. Core re-checks permission as the represented user (`Manage Users`, permission-check class returning 403) and runs the change under that user's rights [C].

2.6 Questions on this pillar

Question	Answer
What is in graph state, and why Redis?	Messages, user context, skills, enabled tools, plan, stage index, step outputs, status. In the Setup agent it is Redis next to the reasoner [O]. TTL and failover behaviour were not found [N]; say so and state what you would require: a TTL longer than the longest plausible approval wait, and a defined behaviour for an in-flight approval on failover
How does a parallel stage fail?	Reads: continue with partial results and report which lookup failed. Writes: fail fast, and report exactly what was done. There is no compensation today [C/I]
Where is the approval pause stored?	The turn ends; approval returns as a new user message; state is in Redis. No expiry was found in V2 [N]. A future runtime auto-declines after 5 minutes and loses the run on restart [O]
Why does `interrupt()` need care?	Resume re-runs the node from the top, so side effects before the interrupt must be idempotent or moved after it. Do not wrap `interrupt()` in a bare `try/except`. Keep interrupt order deterministic
What happens when two things interrupt in parallel?	They resume together by mapping each interrupt id to its value in one `Command(resume=...)`
How do you stop a runaway loop?	Graph recursion limit (config key `recursion_limit`; raises `GraphRecursionError`) plus status-driven routing that has terminal states
How do you test a node?	Call it as a function with a hand-built state; assert on the update. The Executor is pure code, so ordinary unit tests; invariants I-1..I-6 are graph-shape tests enforced by a pre-push hook [O]
Why V2 slower?	More nodes, more LLM calls (about 5 at P95), a thinking phase. Mitigations: incremental planning, a fast path for single-workflow tasks, streaming

2.7 The high-velocity path and the dynamic path (HLD)

- **High-velocity path:** the goal matches one Workflow NLD (for example "Freeze user"). The Reasoner acts as a translator: it reads the pre-authored steps and converts them to a plan.
- **Dynamic synthesis path:** multi-document goals ("clone X and freeze Y"), higher-order journeys, or hybrid plans. The pipeline is `GatherMoreDetails` (clarify) → scaffold ingestion → `compilePlan`.
- **`compilePlan` is two stages:** Stage A is parallel vector search over the action catalog for every scaffold step, giving 3–5 candidate actions each. Stage B is one grounded LLM call that merges steps and wires the data flow. This funnel keeps the model's final decision small as the catalog grows to hundreds of actions.

Chapter 3: Pillar 2, MCP server, Headless 360 and selective tool calling

3.1 What "headless" means, in one defensible paragraph

> Headless means a capability is usable without its UI. Each capability is exposed as an API, discoverable as a tool, and executable over MCP, so an agent on any surface (Setup chat, Slack, a coding agent) calls the same code path with the same permission checks, rather than each surface re-implementing the logic. The UI becomes one client among several.

Your resume phrases it as three stages: **API exposure → tool discovery → MCP execution.** Keep that order.

What is verified [drill2]: the Setup agent is exposed through a near-core proxy that speaks MCP outward and the Agent API inward. The session cache key is `(orgId, mcpSessionId, botId)`. The proxy has a 60 s timeout with 2 retries and a roughly 3-minute servlet and mesh ceiling. A local sample showed p50 about 35 s, p95 about 98 s, p99 about 117 s (directional, not an SLO). The response is text with no UI resource.

What is not verified: the header or token field that carries tenant and user into the agent call; the count of eight onboarded capabilities; the 10+ teams.

3.2 MCP in 90 seconds

MCP is JSON-RPC 2.0 between a **host** (the agent application), a **client** (one connection per server inside the host) and a **server** (exposes capabilities). Lifecycle:

1. Client sends `initialize` with `protocolVersion`, its `capabilities` and `clientInfo`.
2. Server replies with its `protocolVersion`, `capabilities` and `serverInfo`.
3. Client sends the notification `notifications/initialized`. Only then are normal requests allowed.
4. Normal operation: `tools/list`, `tools/call`, `resources/list`, `resources/read`, `prompts/list`, `prompts/get`.
5. Shutdown by closing the transport.

Your `mcp-handshake.json`, corrected (`clientInfo` has a capital I; add the initialized notification; the server declares tools and resources only, so do not list prompts as supported):

```
// client -> server
{ "jsonrpc": "2.0", "id": 0, "method": "initialize",
  "params": { "protocolVersion": "2025-06-18",
              "capabilities": { "sampling": {} },
              "clientInfo": { "name": "my-agent-host", "version": "1.1.0" } } }

// server -> client
{ "jsonrpc": "2.0", "id": 0,
  "result": { "protocolVersion": "2025-06-18",
              "capabilities": { "tools": { "listChanged": true },
                                "resources": { "subscribe": true, "listChanged": true } },
              "serverInfo": { "name": "salesforce-mcp-server", "version": "0.4.1" } } }

// client -> server (notification, no id, no response)
{ "jsonrpc": "2.0", "method": "notifications/initialized" }
```

Tools are described by a name, a description and a JSON Schema `inputSchema`. A call returns `content` blocks and an `isError` flag. Tool failures are reported inside the result (`isError: true`), not as protocol errors, so the model can see and react to them.

3.3 Reference implementation: the server core

Transport-agnostic, so you can explain it regardless of SDK.

```
PROTOCOL = "2025-06-18"

class Session:                                  # one per mcpSessionId
    def __init__(self, org_id, mcp_session_id, bot_id, principal):
        self.key = (org_id, mcp_session_id, bot_id)   # cache key, as in the proxy
        self.principal = principal                    # authenticated user; set by the transport, not
by params
        self.ready = False


def handle(req: dict, s: Session) -> dict | None:
    method, rid = req.get("method"), req.get("id")

    if method == "initialize":
        return ok(rid, {"protocolVersion": PROTOCOL,
                        "capabilities": {"tools": {"listChanged": True}},
                        "serverInfo": {"name": "salesforce-mcp-server", "version": "0.4.1"}})
    if method == "notifications/initialized":
        s.ready = True
        return None                                   # notifications get no response
    if not s.ready:
        return err(rid, -32002, "server not initialized")

    if method == "tools/list":
        return ok(rid, {"tools": [t.describe() for t in visible_tools(s.principal)]})
    if method == "tools/call":
        return ok(rid, call_tool(req["params"], s))
    return err(rid, -32601, f"method not found: {method}")
```

Selective exposure and the call path. Two separate ideas share the word "selective":

```
def visible_tools(principal) -> list[Tool]:
    """Entitlement filter: only advertise what this caller could actually run."""
    return [t for t in REGISTRY.values()
            if t.enabled and t.required_permissions <= principal.permissions]


def call_tool(params: dict, s: Session) -> dict:
    tool = REGISTRY.get(params["name"])
    if tool is None or tool not in visible_tools(s.principal):
        return result_error("unknown or unavailable tool")          # same answer either way: no
probing
    args = params.get("arguments", {})
    try:
        validate(args, tool.input_schema)                           # additionalProperties: false
        out = core.invoke(tool.core_name, args,
                          on_behalf_of=s.principal,                 # identity from the session, never
from args
                          timeout=60)
    except ValidationError as e:
        return result_error(f"invalid arguments: {e.message}")      # model-visible, model can retry
    except CoreError as e:
        return result_error(f"{e.code}: {e.public_message}")
    audit.log(session=s.key, tool=tool.name, user=s.principal.id, status="ok")
    return {"content": [{"type": "text", "text": to_text(out)}], "isError": False}
```

Selective tool calling inside the agent is the second idea and the one with real numbers behind it: retrieval narrows which actions the model may even see. Tool retrieval returns the top 5 candidates above a 0.7 similarity threshold [C], and each selected knowledge document carries a `tools:` list that sets `enabled_tools` [C]:

```
def select_tools(step_text: str, enabled: list[str], k: int = 5, floor: float = 0.7) -> list[str]:
    scored = [(cosine(embed(step_text), t.embedding), t.name) for t in catalog if t.name in enabled]
    return [name for score, name in sorted(scored, reverse=True)[:k] if score >= floor]
```

Say the precision, because drill2 found one coarse tool at the MCP boundary: *at the MCP boundary the agent is exposed as one tool; selection happens inside the graph, where documents enable tools and retrieval narrows them.* Do not imply per-entity MCP tools unless that is true.

3.4 Onboarding an entity to Headless 360: the checklist

From the HLD and your resume. For each Setup entity:

1. **API exposure:** the entity must be publicly exposed through the Data API or Tooling API (an HLD requirement for API grounding) [HLD].
2. **Grounding document:** an API-grounding document per MetadataType, with schema, API support and example usage [HLD].
3. **Skill/NLD with metadata:** operations, required permissions, related documents, scopes, and the `tools:` list [HLD].
4. **Tool contract:** input and output JSON Schema with `additionalProperties` false; operation mapped to an allowed verb (the metadata write map for `Group` is CREATE / READ / UPDATE only) [C].
5. **Permission mapping:** the user permission the action requires (for example Manage Users) and the surface blocklist, so it is reachable through the agent path only [C].
6. **Confirmation policy:** `requireUserConfirmation` for writes, draft-then-commit for bulk changes [C].

7. **Registration:** add to the allowlist with a named owner [C].
8. **Eval cases:** utterances plus a post-condition check (for example a query that confirms the group exists) [O].

A descriptor you can sketch on a whiteboard:

```
entity: PublicGroup
api: { family: data, object: Group }
operations:
  - name: create
    tool: CreateGroup
    permissions: [ManageUsers]
    confirmation: required
    input_schema: schemas/create_group.input.json      # additionalProperties: false
    output_schema: schemas/create_group.output.json
  - name: add_members
    tool: ManageUserAssignments
    mode: draft_then_commit
    limits: { max_assignments: 50 }
surfaces: { rest: blocked, apex: blocked, agent: allowed }
grounding: [docs/api/group.md, docs/nld/create_public_group.md]
evals: [create_public_group_basic, create_public_group_duplicate_name]
```

3.5 Questions on this pillar

Question	Answer
Why MCP rather than direct REST per surface?	One discoverable, typed contract any agent host can call; the permission and audit path stays in one place. Cost: one more hop, and the proxy adds latency (p50 about 35 s end to end in the local sample, dominated by the agent turn, not MCP)
What identity does the call run as?	The requester, via the authenticated session. Drill2 did not read the header that carries user identity into the agent call. Know the failure case: if that header carried an integration user instead, the permission check would be the integration user's, which is a confused-deputy bug. The test is to call as a low-privilege user and confirm a 403
How do sessions work?	Key is `(orgId, mcpSessionId, botId)`; the org is part of the key so one tenant's session cannot be reached by another
Timeouts and retries?	60 s with 2 retries at the proxy, a roughly 3-minute ceiling. Retries are only safe because the call into the agent is a conversation turn, not a raw write; the write path itself has no retries [O]
What if two clients call at once?	Sessions are independent; within a session, turns are serialised by the session engine
How do you handle `listChanged`?	The server advertises it so a client re-lists tools when entitlements or the catalog change
How do you keep a poisoned tool result from steering the agent?	Tool output is data. Plan steps and `$ref` wiring are produced and checked by code, tools come from an allowlist, and writes need approval
Named the eight?	Either name them from memory or describe the onboarding mechanism without the count. Do not give a number you cannot reconstruct

3.6 Runnable reference server, selective tool calling, and the issues

This is an **illustrative reimplementation**, not the production source. Say so. Shipped facts are in the table in 3.6.0; everything else is design you would do next.

3.6.0 Honesty frame (say this first)

> "Here's a small reference version of the pattern. The shipped version was simpler: one coarse tool wrapping the agent, behind a proxy. I'll tell you which parts are shipped and which are what I'd do next."

Shipped (defend)	Design only (label it)
Agent exposed over MCP via near-core proxy	Per-tool exposure (`find_tools`)
Session key `(orgId, mcpSessionId, botId)`	Identity from headers only
60 s proxy timeout, 2 retries	Write confirmation + idempotency key
Retrieval top 5 @ 0.7 → `enabled_tools` from the skill doc's `tools:` list	Per-user tool permissions
Local latency sample p50 ~35 s, p95 ~98 s	

Do **not** claim: "8 entities across 10+ teams" (not found in the drill), any resume metric, or that the user-identity header was read (it was not).

3.6.1 60-second architecture answer

MCP is JSON-RPC 2.0: `initialize` (negotiate version + capabilities: `tools.listChanged`, `resources.subscribe`) → `tools/list` → `tools/call`. The host (the LLM client) sees tool names and schemas only. My server turns those calls into the agent. Session state lives server-side under `(org, mcp-session, bot)` so a multi-turn conversation keeps its plan and pending approvals.

3.6.2 Selective tool calling

Problem: with N tools in the prompt, the model picks the wrong one more often, token cost grows, and risky write tools sit next to harmless reads. **Mechanism (shipped):** embed the request, retrieve the top 5 skill docs at ≥ 0.7, union their `tools:` lists into `enabled_tools`. The plan is then validated against that set, and a step naming a tool outside it is rejected (`validate_plan` in HM_ROUND_PREP.md). **Key line:** the allowlist is enforced in code at validation time, not only by the model's good behavior.

Issues with it (the interview gold)

1. **Threshold calibration.** My reference test: the query `"user"` matched nothing at 0.7 while the full sentence matched. Phrasing sensitivity is real. There was no calibrated confidence gate. Fix: build a labelled set of (request → correct tools), sweep the threshold, and report recall@k.
2. **Recall vs precision asymmetry.** A missing tool = task fails (visible). An extra tool = slightly worse accuracy (silent). So bias toward recall for reads, precision for writes.
3. **Empty set behavior.** When nothing clears the threshold, do not fall back to "all tools." That defeats the point. Return a clear "can't do that" or ask a clarifying question.

4. **Multi-step requests.** "Why can't Priya access X?" needs FindUser → FindPermissionSets → FindPublicGroups → FindQueues. Retrieval on the *whole utterance* can miss the later steps; docs listing co-required tools in `tools:` is the workaround, and it is manual and drifts.
5. **Stale index.** Tools change, the embedding index doesn't. Needs re-index on deploy, and a test that every tool in the catalog is reachable by at least one doc.
6. **Retrieval is not authorization.** Narrowing the menu is a relevance feature. Who may call a write tool is a separate permission check against the user's identity.

3.6.3 MCP server challenges

1. **Coarse tool vs many tools.** One `ask_agent` tool is easy to onboard but opaque: no per-step approval, no per-step authz, the client LLM can't reason about steps, and latency is the whole run. Many fine tools give control but bloat the client's context. Middle path: a few task-level tools plus `find_tools`.
2. **Latency vs timeout.** p95 ~98 s against a 60 s proxy timeout means a meaningful tail times out, and then **retries re-run a long agent**. Fixes: stream progress notifications, or return a task id and poll (async task pattern), and keep the timeout above the p95 or cut the p95.
3. **Retries and side effects.** Retrying a timed-out write can apply it twice. Retry reads only, or require an idempotency key (see `confirm_action`). The server cannot know whether the first attempt landed.
4. **Identity.** Tool *arguments* are model-controlled, so a prompt-injected `org_id` is an authz bypass. Identity must come from the trusted transport (headers set by the proxy). Weak spot: in the shipped version the user-identity header was never read, so be ready to say that plainly and what the fix is.
5. **Prompt injection through tool results.** Setup data (user names, descriptions) is attacker-influenceable text that flows back to the model. Treat results as data, keep writes behind confirmation, and keep the confirmation message deterministic (not LLM-written).
6. **Session state.** In-memory `SESSIONS` dies on restart and breaks with >1 replica. Needs a shared store (Redis/DB) with a TTL, or sticky routing. Memory grows without eviction.
7. **Version/capability drift.** `listChanged` means clients may cache stale tool lists. Version the tools; never change a schema in place.
8. **Observability.** Log per call: session key, tool, enabled set, retrieval scores, latency, outcome. Without scores you can't debug "why did it not offer tool X."
9. **SDK churn (hit this today).** `mcp` 2.x renamed `FastMCP` to `MCPServer` and broke v1 code. Pin the version. Good small story about dependency pinning.

3.6.4 Likely questions → one-line answers

- *Why MCP and not direct function calling?* Standard protocol so many hosts reuse one server; discovery and capability negotiation come built-in. Cost: extra hop and latency.
- *stdio vs HTTP?* stdio for local single-user tools; streamable HTTP behind a proxy for multi-tenant, which is why the session key exists.
- *How do you stop the model calling a write tool unprompted?* Enabled-set allowlist + confirmation step for `writes: True` + identity-based permission check.
- *How would you evaluate selective calling?* Labelled requests, recall@k, wrong-tool rate with/without narrowing, end-to-end task success.
- *What would you change first?* Read the identity header and enforce per-user tool permissions; then async/streaming for the tail latency.

3.6.5 The reference code (read it once, then retype `select_tools` and `confirm_action` from memory)

```
"""
Reference MCP server for interview discussion: a "setup agent" exposed over MCP.

This is an ILLUSTRATIVE reimplementation of the shape of the work, not the production code.
Say so in the interview. Production facts you can defend (from your drill notes):
  - the agent was exposed through a near-core proxy, session key (orgId, mcpSessionId, botId)
  - proxy timeout 60s with 2 retries
  - tool retrieval: top 5 at similarity >= 0.7; skill docs carry a `tools:` list -> enabled_tools
  - the first shipped version was ONE coarse tool wrapping the whole agent
What this file adds on top, and is labelled as design (not shipped): per-tool exposure,
identity from headers, write confirmation, and idempotency.

Run:  uv run --with "mcp<2" python mcp_server_reference.py          (stdio)
"""
from __future__ import annotations

import asyncio
import re
import uuid
from dataclasses import dataclass, field

from mcp.server.fastmcp import Context, FastMCP

mcp = FastMCP("setup-agent-mcp")

# ---------------------------------------------------------------------------- tool catalog
# Fine-grained tools the agent can use internally. `writes` marks side effects.
CATALOG: dict[str, dict] = {
    "FindUser":            {"desc": "look up a user by name or email",            "writes": False},
    "FindPermissionSets":  {"desc": "list permission sets assigned to a user",    "writes": False},
    "FindPublicGroups":    {"desc": "list public groups a user belongs to",       "writes": False},
    "FindQueues":          {"desc": "list queues a user belongs to",              "writes": False},
    "AssignPermissionSet": {"desc": "assign a permission set to a user",          "writes": True},
    "DeactivateUser":      {"desc": "deactivate a user account",                  "writes": True},
}

TOP_K = 5
THRESHOLD = 0.7


def _tokens(s: str) -> set[str]:
    return set(re.findall(r"[a-z]+", s.lower()))


def score(query: str, desc: str) -> float:
    """Stand-in for embedding cosine similarity (Jaccard on tokens). Swap for a real embedder."""
    q, d = _tokens(query), _tokens(desc)
    return len(q & d) / len(q | d) if q | d else 0.0


def select_tools(query: str, k: int = TOP_K, threshold: float = THRESHOLD) -> list[str]:
    """Selective tool calling: narrow the menu BEFORE the LLM sees it.

    Fails open on nothing: if no tool clears the threshold we return [] and the caller must
    say 'I can't do that' rather than hand the model all tools (that is the failure mode
    selective calling exists to prevent).
    """
    ranked = sorted(CATALOG, key=lambda n: score(query, CATALOG[n]["desc"] + " " + n), reverse=True)
    return [n for n in ranked[:k] if score(query, CATALOG[n]["desc"] + " " + n) >= threshold]


# ---------------------------------------------------------------------------- session state
@dataclass
class Session:
    org_id: str
    user_id: str
    enabled: set[str] = field(default_factory=set)
    pending_writes: dict[str, dict] = field(default_factory=dict)   # confirmation_id -> action
    done_keys: dict[str, dict] = field(default_factory=dict)        # idempotency_key -> result
```

```
SESSIONS: dict[tuple[str, str, str], Session] = {}   # (org_id, mcp_session_id, bot_id)


def identity(ctx: Context) -> tuple[str, str, str]:
    """DESIGN: derive identity from transport headers, never from tool arguments.

    The model controls tool arguments, so an `org_id` argument is attacker-controlled via
    prompt injection. Headers are set by the trusted proxy. (In the shipped version the
    user-identity header was not read - own that if asked.)
    """
    req = ctx.request_context.request          # present on HTTP transports; None on stdio
    h = req.headers if req is not None else {}
    org = h.get("x-org-id", "dev-org")
    user = h.get("x-sfdc-user-id", "dev-user")
    sid = h.get("mcp-session-id", "stdio")
    return org, user, sid


def get_session(ctx: Context, bot_id: str = "setup-bot") -> Session:
    org, user, sid = identity(ctx)
    return SESSIONS.setdefault((org, sid, bot_id), Session(org_id=org, user_id=user))


# ---------------------------------------------------------------------------- tool execution
async def run_tool(name: str, args: dict) -> dict:
    """Stub for the real backend call (proxy -> core). Real one: 60s timeout, 2 retries,
    retry ONLY reads - a retried write without an idempotency key can double-apply."""
    await asyncio.sleep(0)
    return {"tool": name, "args": args, "ok": True}


# ---------------------------------------------------------------------------- MCP surface
@mcp.tool()
async def ask_setup_agent(utterance: str, ctx: Context) -> dict:
    """COARSE tool (what shipped first): hand the whole request to the agent.

    Pro: tiny surface, the agent keeps planning internally, easy to onboard teams.
    Con: the calling model gets no visibility, no per-step permissions or approvals,
    and latency is the whole agent run (p50 ~35s, p95 ~98s in the local sample),
    which is uncomfortably close to a 60s proxy timeout.
    """
    s = get_session(ctx)
    s.enabled = set(select_tools(utterance))
    if not s.enabled:
        return {"status": "no_capable_tool", "message": "No setup tool matches this request."}

    results = []
    for name in sorted(s.enabled):
        if CATALOG[name]["writes"]:
            cid = str(uuid.uuid4())
            s.pending_writes[cid] = {"tool": name, "utterance": utterance}
            results.append({"tool": name, "status": "needs_confirmation", "confirmation_id": cid})
        else:
            results.append(await run_tool(name, {"utterance": utterance}))
    return {"status": "ok", "enabled_tools": sorted(s.enabled), "results": results}


@mcp.tool()
async def find_tools(query: str, ctx: Context) -> list[dict]:
    """Fine-grained alternative: let the client discover which tools apply to a request
    (retrieval-based narrowing exposed as a tool) instead of dumping the full catalog."""
    return [{"name": n, "description": CATALOG[n]["desc"], "writes": CATALOG[n]["writes"]}
            for n in select_tools(query)]


@mcp.tool()
async def confirm_action(confirmation_id: str, idempotency_key: str, ctx: Context) -> dict:
    """Execute a pending write after explicit user approval. Idempotent on `idempotency_key`
```

```
    so a retry after a timeout cannot apply the write twice."""
    s = get_session(ctx)
    if idempotency_key in s.done_keys:
        return s.done_keys[idempotency_key]
    action = s.pending_writes.pop(confirmation_id, None)
    if action is None:
        return {"status": "error", "message": "unknown or expired confirmation_id"}
    result = await run_tool(action["tool"], {"utterance": action["utterance"]})
    s.done_keys[idempotency_key] = result
    return result


if __name__ == "__main__":
    mcp.run()   # stdio; use mcp.run(transport="streamable-http") behind the proxy
```

Chapter 4: Pillar 3, retrieval and grounding with connectors

4.1 The retrieval path [C/O]

```
Knowledge node -> QueryDataSource action -> planner -> Core
  Core runs three branches in parallel: product docs | NLD | navigation (each behind a gate)
  NLD branch: C360Retriever -> POST /retrievers/{name}/text-retrieval (AI Gateway, 35 s)
    -> Data Cloud Query Federator -> hybrid_search(...) on the index  (pre-filter inside the call)
    -> reranker (up to 320 chunks, 12 s; failure keeps original order)
  Core post-filters: permissions, client, de-duplication
  LLM selects documents -> FilterKnowledgeDocuments keeps only those
  Selected documents' `tools:` -> enabled_tools
```

Parameters: topK default 10 (max 2048); tool retrieval top 5 at 0.7; token exchange into the knowledge tenant with one retry after refresh on 401/403; a failed branch returns nothing, so the agent continues with fewer documents or no tools (silent degradation).

4.2 Reference implementation: connectors, retrieval and grounding

A connector is one retriever behind a common interface, so branches can be added without touching the graph:

```
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import dataclass, field
from typing import Protocol


@dataclass
class Chunk:
    id: str
    text: str
    score: float
    source: str                       # which connector returned it
    required_permissions: set[str] = field(default_factory=set)
    tools: list[str] = field(default_factory=list)


class Retriever(Protocol):
    name: str
    def search(self, query: str, *, filters: dict, top_k: int, timeout: float) -> list[Chunk]: ...


class DataCloudRetriever:
    """Hybrid (vector + keyword) retriever over a Data Cloud index, behind the AI Gateway."""
    def __init__(self, name, retriever_name, gateway, token_provider):
        self.name, self.retriever_name = name, retriever_name
        self.gateway, self.tokens = gateway, token_provider

    def search(self, query, *, filters, top_k, timeout):
        token = self.tokens.exchange()                    # org JWT -> knowledge-tenant token
        resp = self.gateway.post(
            f"/retrievers/{self.retriever_name}/text-retrieval",
            json={"query": query, "topK": top_k, "filter": filters},   # filter runs inside the search
engine
            headers={"Authorization": f"Bearer {token}"}, timeout=timeout)
        if resp.status in (401, 403):                     # one retry after a token refresh
            token = self.tokens.exchange(force=True)
            resp = self.gateway.post(..., headers={"Authorization": f"Bearer {token}"},
timeout=timeout)
        resp.raise_for_status()
        return [to_chunk(r, source=self.name) for r in resp.json()["results"]]
```

The orchestrator runs branches in parallel, never lets one branch take the others down, and records what degraded:

```
@dataclass
class RetrievalResult:
    chunks: list[Chunk]
    degraded: list[str]                     # connectors that failed or timed out


def retrieve(query: str, user, connectors: list[Retriever], *, top_k=10, floor=0.0, timeout=35.0):
    results, degraded = [], []
    with ThreadPoolExecutor(max_workers=len(connectors)) as pool:
        futures = {pool.submit(c.search, query, filters=prefilter(user, c), top_k=top_k,
timeout=timeout): c
                   for c in connectors}
        for fut in as_completed(futures):
            c = futures[fut]
            try:
                results.extend(fut.result())
            except Exception:
                degraded.append(c.name)       # empty branch, but visible, not silent

    results = dedupe(results)
    results = rerank(query, results, limit=320, timeout=12)       # on failure: keep original order
    results = [c for c in results                                  # post-filter: caller's permissions
               if c.required_permissions <= user.permissions and c.score >= floor]
    return RetrievalResult(chunks=results[:top_k], degraded=degraded)
```

Grounding means the answer can be traced to retrieved chunks, and anything that cannot is flagged:

```
def answer_with_grounding(question: str, result: RetrievalResult, llm) -> dict:
    if not result.chunks:                                  # fail closed: do not improvise
        return {"answer": "I could not find reliable guidance for that.",
                "grounded": False, "degraded": result.degraded}

    sources = {f"S{i+1}": c for i, c in enumerate(result.chunks)}
    prompt = build_prompt(question, sources)               # numbered sources, "cite as [S#], use only
these"
    draft = llm.invoke(prompt)

    cited = set(re.findall(r"\[(S\d+)\]", draft.text))
    invalid = cited - sources.keys()                       # model cited something that was not
provided
    grounded = bool(cited) and not invalid and supported(draft.text, [sources[c] for c in cited])
    return {"answer": draft.text, "citations": sorted(cited), "grounded": grounded,
            "degraded": result.degraded}
```

What this mirrors in the real system: the planner trace records an `OutputEvaluationStep` with `GROUNDED` / `UNGROUNDED` [O]; the Trust Layer adds PII, toxicity and prompt-defence checks at about 2 s each [O]. What it does not mirror: **there is no calibrated confidence score that gates a side effect** [N]. The `ClassificationConstants` thresholds in Core are dead code [C]. Do not claim a confidence gate; say what you would build from verifiable signals: entities resolved, inputs bound, schema-valid, document matched.

For ResolveFlow, the retrieval gate is yours and is real: a measured relevance threshold decides whether retrieved evidence may influence downstream reasoning (about 2,750 closed-issue chunks).

4.3 DLO and DMO, answered properly

Term	Meaning
Data stream	The feed that brings a source (S3 here) into Data Cloud
DLO (data lake object)	The raw landed copy of the source, in the source's own shape. Nothing is renamed or reconciled yet
DMO (data model object)	The canonical, harmonised object, produced by mapping DLO fields onto the shared data model. Downstream (identity, segments, search) uses DMOs
mDLO / mDMO	The HLD's metadata variants for NLDs and API grounding, fed by a metadata-extraction framework with a status machine (New → Processing → Syncing → Data_Copied → Ingestion_Requested → Ingestion_Complete) [HLD]
Chunk DMO, embedding, index	Chunking splits documents into passages; each gets an embedding vector; the index is hybrid (vector + keyword); the retriever is a named, configured query over that index

The Setup knowledge pipeline as found in code [O]: NLD Markdown in Core (LLM-drafted, human-reviewed) → CSV in S3 (us-east-2) → data stream `Agentforce NLD S3` → DLO `Agentforce_NLD_S3__dlm` → DMO → chunk DMO → embedding → hybrid index `…_index__dlm` → retriever.

Say this about the difference between the doc and the code: **the HLD describes mDLO ingestion with a status machine run periodically; the code shows manual, skill-driven ingestion and no mDLO in this path** [N]. Ingestion is manual with no schedule; a corpus change also needs a Core jar version bump of roughly 12–24 hours. The Setup corpus is **one shared tenant** (`agentforcenld`) holding no customer data, so tenant isolation for knowledge means authenticating the calling org through token claims, not separating data. Customer-data retrievers are isolated per org and data space through signed claims (`NO_DATASPACE_ACCESS` for unauthorised users) [C].

Pre-filter versus post-filter: the filter expression is an argument to `hybrid_search` and runs inside the engine (cheap and safe); post-filters run after retrieval (rerank, Core permissions, dedupe, LLM selection) and can leave too few results, which is why recall matters.

Not found, so do not state: production chunk size and overlap, the production embedding model, fusion weights, score thresholds. Harness defaults were 512-token passages, a 1024-dimension embedding with cosine, HNSW.

4.4 Questions on this pillar

Question	Answer
What is the weakest part of retrieval?	Top-5 recall for tool selection: if the right action is not in the five, the model cannot choose it and no fallback was found. Second, silent degradation when a branch returns empty
How do you evaluate retrieval?	recall@k on labelled utterances → expected document, tool-selection accuracy, then end-to-end golden tasks. Replay production failures as cases
Why does retrieval double as a permissions concern?	Selected documents' `tools:` become the enabled tools, so a wrong document can widen what the agent may call. Hence the allowlist and in-Core checks sit behind it

Question	Answer
What if the index is stale?	Freshness is bounded by manual refresh plus the jar bump. Fix: scheduled ingestion, a versioned corpus, an eval gate before swapping the index, and a canary index
Pre- or post-filter for permissions?	Pre-filter where the metadata allows, because it cannot leak. Post-filter as a backstop
What is a connector here?	A named retriever behind a common interface (docs, NLD, navigation, org metadata grounding); add one without touching the graph

Chapter 5: "Flow of a production agent", infrastructure, and the other questions you were asked

5.1 The flow of a production agent: the six layers

When asked "tell me the flow of a production-based AI agent", answer in layers, then name what makes it production-grade:

1. **Surface and identity.** The client authenticates; a bootstrap step issues a short-lived org-scoped JWT. Session state is created.
2. **Gateway and session.** An API front door checks the token, rate-limits and opens a stream (SSE). A session engine owns multi-turn state.
3. **Orchestration.** A planner service relays to the reasoning service, which runs a graph: classify → retrieve → plan → approve → execute.
4. **Model access.** Every LLM call goes through one gateway: provider failover, rate limits, Trust Layer checks (PII, toxicity, prompt defence), regional rules, audit.
5. **Knowledge and tools.** Retrieval through a gateway into the knowledge store; tool execution through a planner into the system of record, as the user.
6. **Governance around it.** Human approval layers, evals, tracing, alerts, staged rollout and rollback.

Then the production-grade points: authorization is deterministic code, not model output; writes sit behind approval; failures are attributable to one step; timeouts and retries are set per hop (and you can name where they are wrong); there is a fallback to the previous graph; changes ship behind a weekly promotion and feature gates.

Close with the honest weak spots. They read as seniority: no action retries until idempotency exists, timeouts that do not line up (false error on turns of about 5 minutes), partial writes, no eval release gate, no SLOs.

5.2 One request through every hop

Request: "Create a public group Sales Ops and add users X and Y."

```
Browser (Setup chat)
 1  GET /agentforce/bootstrap ..... Core issues short-lived OrgJWT
 2  POST .../sessions/{id}/messages/stream (SSE, Bearer OrgJWT) -> Agent API   [stream cap 601 s]
 3  mesh mTLS -> session worker (state in Redis)
 4  SSE -> planner (bot-svc-llmagent)
 5  mesh mTLS -> reasoner (graph: Knowledge, Assistant, Executor, Discovery)  [200 req/org/min]
 6  every LLM step -> LLM Gateway -> OpenAI / Azure / Anthropic / Bedrock     [retries 10x, Trust
Layer]
 7  Knowledge: QueryDataSource -> planner -> Core /support/functions          [60 s, no retry]
 8  Core C360Retriever -> AI Gateway -> Data Cloud hybrid search -> rerank
 9  Assistant: todo plan (2 tasks) -> turn ends -> user approves (new message)
10  Executor: createGroup -> planner (deterministic confirmation) -> Core      [60 s, no retry]
11  Core: Manage Users check -> validator -> permission check -> Group.save()  as the user
12  ManageUserAssignments: draft (commit=false) -> user confirms -> commit (max 50, bulk save)
13  result card streams back over SSE
```

The write happens only at steps 11 and 12, inside Core, as the user. Consent appears three times: plan approval, tool confirmation, draft-then-commit. Control never goes from the model straight to the database.

5.3 Where it runs and how it ships

Deployable	Runs on	Ships by	Rolls back by
Agent YAML, NLDs, actions	Core monolith (in the Core jar)	Core release train; graph promoted every Tuesday; per-org and per-user gates	Per-org switch to the `Stable` graph. **Reverts the graph only**, not actions, templates or NLDs
Agent API, session worker	Internal runtime on AWS (Falcon)	Strata CI → Docker → Helm + Terraform; staged promotion with Argo canary	Argo / Helm rollback
Planner	AI Cloud (Falcon)	Soak on one instance, then broader; blue/gold releases; frozen around Dreamforce	Switch blue/gold; Argo rollback
Reasoner	AI Cloud (Falcon)	Mesh-staggered rollout (about 21 instances)	Argo rollback
LLM Gateway	AI Cloud (Falcon)	Health-check analysis during rollout; 1-minute pre-rollout pause	Automated on failed analysis. HPA 3–9, 4 CPU / 12 Gi
AI Gateway	AI Cloud (Falcon)	Falcon Helm; fit tests	Helm rollback. HPA 3–12
Knowledge index	Data Cloud, one shared tenant	Manual: S3 upload, index rebuild on 4 orgs, jar bump	Re-upload and rebuild [I]

First-principles answers for the parts you may not own (say "the platform does X; the principle is Y"):

- **Container and process model.** A Python FastAPI service in a Docker image, served by gunicorn with uvicorn workers; graceful drain on SIGTERM so in-flight turns finish before the pod exits. The drill verified the image, gunicorn config and drain chain [O].
- **Scaling.** Stateless pods behind an HPA, state in Redis, per-org rate limit. LLM calls dominate cost and latency, so token and call budgets matter more than CPU.
- **Canary and rollback.** Send a slice of traffic to the new version, watch error rate and latency, and promote or revert automatically on failed analysis. Blue/gold swaps traffic between two environments for instant rollback.
- **Why feature gates as well.** The graph ships with the Core release to every instance and is enabled per org and per user, so rollout is by who, not by traffic percentage.
- **Observability.** Per-layer logs, planner trace steps (UserInput, LLM, Function, Transition, PlannerResponse, OutputEvaluation), session tracing, a trace id across hops, OpenTelemetry sampled at 1% in the gateway [O]. Alerts exist for the retriever (Sev2 under 99.7%, Sev1 under 99.3%, Slack only), the planner, and the Data Cloud federator. None found for the LLM Gateway, reasoner or the Setup agent; no SLOs [N].

5.4 Failure diagnosis (symptom → layer)

Symptom	Layer	First question
Chat will not open	Token / session	Did bootstrap issue a JWT? Is planner provisioning stuck in `ENQUEUED`?
"Something went wrong" but the change happened	Timeout chain	Did the turn run 4.5–5 minutes? Backend success versus the 601 s stream cap and 300 s mesh timeout

Symptom	Layer	First question
Vague answer / wrong workflow	Retrieval	Did the document branch return anything? Any `DEADLINE_EXCEEDED`?
Wrong action chosen	Tool selection	Was the right action in the top 5 above 0.7?
403 from an action	Permission / identity	Does the user hold Manage Users? Was the JWT valid?
Group exists, members missing	Partial failure	Did the commit step fail or time out?
Everything slow at once	Gateway / provider	Is the LLM Gateway degraded or the org rate-limited (429)?
Outdated instructions	Knowledge freshness	When was the corpus last uploaded and the index rebuilt?

Template for a "challenging problem" story: name the layer, the mechanism, the evidence that confirmed it, the fix, and the regression test you added. Pick stories from work you can prove.

Chapter 6: Pillar 4, the resume, line by line

Your drills checked these. Use the table to decide what to say, not what to claim.

Resume line	Drill result	What to do
Five workflows, about 60 days → 5 days (about 92%)	Number not found; five not named	Write the method line below. Name your five from memory
Retrieval-to-action architecture, versioned Skills, permission-aware tools	Strong, verified	Know chapters 2–5 cold
Eight capabilities, 10+ teams, Headless 360	Not found	Name the eight or give the mechanism only (3.4)
Slack agent: Python and TypeScript, Bolt, HITL, 150+ users, 20 → under 3 min	Only the MCP path found, no Bolt app	Know exactly which repo is yours. Know the Slack 3-second acknowledgement, the Slack-to-Salesforce user mapping, and the Block Kit approval flow
Evaluation framework	Defensible	524 utterances, 81.7% pass, 88.3% stable, κ 0.649, no CI gate
AWS, Docker, Kubernetes, CI/CD, Splunk, about 40% throughput	Mostly not found	Use 5.3. Do not substitute "50% faster" for "40% throughput": it measures latency, not throughput
ResolveFlow 50 scenarios, 86% routing accuracy, 24% lower cost	Public repo shows 11 safety tasks and 6 capability checks, OpenAI only	Unresolved from Oct 2. Decide the wording before the round

Number defence line. Fill one per number; if you cannot, drop the number and keep the mechanism:

> "[Number] is [metric], measured from [start event] to [end event] in [system], over [window or N requests], [median or mean]. The caveat is [what else changed]."

Example of a mechanism-only answer: "*It took access requests from a multi-week manual queue to self-service in days; the main lever was removing the manual triage-and-implement handoff.*"

Slack agent flow to be able to narrate: Slack event → acknowledge within 3 seconds → map the Slack user to the Salesforce user → agent call → approval block → click → execute under that user's identity → update the message. The resume says "runs under the requesting user's own permissions", so know exactly how identity is established at each hop.

Evals: layers are retrieval recall, tool-selection accuracy, plan validity, permission enforcement (negative tests: a user without access must be refused), structured-output schema checks, failure paths, and replayed production failures. Gate: block promotion on a per-category pass-rate floor.

Chapter 7: LangGraph fundamentals

Source: the LangGraph documentation pages for the graph API, persistence and interrupts, fetched 2026-10-12. Items marked *(general knowledge)* were not in the fetched pages; check them against the version you pin.

7.1 What it is, and how it differs from LangChain

LangGraph is a low-level orchestration framework and runtime for long-running, stateful agents. It lets you mix deterministic code and LLM steps in one graph. Your V1 → V2 story in these terms:

	LangChain (V1 style)	**LangGraph (V2 style)**
Shape	Chains (a straight pipeline) and agent loops where the model chooses each next step	An explicit graph of nodes and edges, with cycles allowed
Control flow	Inside the model's head	In your code: edges, conditional routing, `Command`
State	Passed implicitly between steps	A typed shared state with reducers
Pause and resume	Hard	First-class: checkpointers plus `interrupt()`
Testing	Whole loop	Node by node

7.2 State, reducers, nodes, edges

- **State schema** is a `TypedDict` (or a dataclass for defaults, or a Pydantic model for validation, which is slower). Pass separate `input_schema` and `output_schema` to constrain what the graph accepts and returns.
- **Reducers** merge a node's update into state: a binary function `(left, right)`. Without a reducer a key is overwritten. Use `Annotated[list, operator.add]` to append, `add_messages` or `MessagesState` for message lists, and `Overwrite([...])` to clear a merged field. **When parallel nodes write the same key in one super-step, that key needs a reducer.**
- **Node:** a function taking state and returning a partial update; it can also take `config` and `runtime`.
- **Edges:** `add_edge(a, b)`; `START` and `END`; `add_conditional_edges(node, router[, mapping])` where the router returns a node name or a list of names. A node should use either static edges or dynamic routing (conditional edges or `Command`), not both.
- **`Command(update=..., goto=...)`** updates state and routes in one return value. Annotate the return type with the possible targets. `Command(graph=Command.PARENT, ...)` jumps to a node in the parent graph from a subgraph.
- **`Send(node, payload)`** is dynamic fan-out: return a list of `Send` objects and each runs the target node with its own payload, in parallel, with a number decided at runtime. This is the map-reduce primitive and what the parallel stage in chapter 2 uses.
- **`compile()`** produces the runnable graph. Options include `checkpointer`, `store`, `cache` (with a per-node `CachePolicy`) and breakpoints.
- **Recursion limit:** top-level config key `recursion_limit` (default 1000 from version 1.0.6); exceeding it raises `GraphRecursionError`. `RemainingSteps` lets a node route away before the limit.

- **Subgraphs:** a compiled graph used as a node. A key shared between parent and subgraph state needs a reducer on the parent. *(Docs page did not show construction; check the subgraphs page.)*

Execution model, so you can explain "super-steps": the graph runs in discrete steps. All nodes scheduled for a step run (in parallel), their updates are applied through the reducers, then the next step is computed. A checkpoint is written at each step boundary.

7.3 Persistence: threads and checkpointers

- A **thread** is one conversation, selected with `config={"configurable": {"thread_id": "..."}}`.
- A **checkpointer** persists the graph's state at every step as checkpoints: short-term, thread-scoped memory. It is what enables resume after a pause or a crash, and inspecting or modifying state.
- Checkpointer classes named in the docs: `InMemorySaver` (RAM; lost on restart), `SqliteSaver` (local file; development), `PostgresSaver` and `AsyncPostgresSaver` (call `.setup()` once; thread ids must stay under 255 characters). The Setup agent keeps graph state in Redis [O]; ResolveFlow uses Postgres, which is your own project.
- A **store** (`InMemoryStore`, or a durable one) is separate: long-term, cross-thread memory such as user preferences, outside graph state.
- Checkpoints accumulate on long conversations; prune them on a schedule.
- *(General knowledge)* `graph.get_state(config)` returns a snapshot with the next nodes; `graph.update_state(...)` edits state; replaying from an earlier checkpoint is "time travel." Verify names against your version.

7.4 Human in the loop: `interrupt()`

```
from langgraph.types import interrupt, Command

def approval_node(state):
    ok = interrupt({"question": "Proceed?", "details": state["details"]})   # pauses here
    return Command(goto="proceed" if ok else "cancel")

config = {"configurable": {"thread_id": "t1"}}
graph.invoke(inputs, config)                      # returns with result["__interrupt__"]
graph.invoke(Command(resume=True), config)        # same thread_id; resume value becomes interrupt()'s
return
```

Requirements: a checkpointer (durable in production), a `thread_id`, and a JSON-serialisable payload. `interrupt(value, response_schema=None)` can validate the resume value against a Pydantic model, `TypedDict` or dataclass (version 1.2.12 and later).

The pitfalls an interviewer who knows LangGraph will probe:

1. **Resume re-runs the node from its first line.** Everything before `interrupt()` runs again. Keep it side-effect free, or make it idempotent, or put the side effect after the interrupt or in another node.
2. **Never wrap `interrupt()` in a bare `try/except`.** The pause is implemented as an exception; a broad handler swallows it.
3. **Keep interrupt order deterministic.** Matching is index-based within a node. Do not skip interrupts conditionally or loop over data whose length can change between runs.
4. **No `while True` around `interrupt()`.** Each resume replays earlier iterations. Use a conditional edge back to the node for validation loops.
5. **Parallel interrupts** resume together: pass `Command(resume={interrupt_id: value, ...})`.

6. **Static breakpoints** (`interrupt_before`, `interrupt_after`) are for debugging, not for production approval flows.
7. Subgraphs restart both the parent node and the subgraph node on resume.

Link to your design: the Setup agent implements the pause as an ended turn plus approval arriving as a new message with state in Redis, which is the same idea without `interrupt()`; a LangGraph rebuild would use the checkpointer plus `interrupt()`. ResolveFlow uses `interrupt()` with state in Postgres.

7.5 Streaming

(General knowledge, verify against your version.) `graph.stream()` / `astream()` take a `stream_mode`: `values` (full state after each step), `updates` (each node's delta), `messages` (LLM tokens as they are produced), `custom` (arbitrary events a node emits), `debug`. The Setup stack streams over SSE end to end; the HLD's "thinking out loud" proposal is intermediate plan streaming, which is a proposal, not shipped.

7.6 Questions you may get on LangGraph itself

Question	Answer
Node versus edge versus state?	Node: unit of work returning a partial update. Edge: allowed transition (static or conditional). State: typed shared data merged by reducers
Why do reducers matter?	Without one, the last writer wins; parallel writers to one key need a merge function
What is a checkpoint?	A snapshot of state at a step boundary for a thread; the basis of resume, inspection and replay
`Command` versus conditional edge?	`Command` updates state and routes together from inside the node; conditional edges route from a function over state. Do not use both on one node
How would you do parallel tool calls?	`Send` per step, a reducer on the results key, and a join node that runs once after all branches complete
How do you bound a loop?	`recursion_limit`, terminal statuses and a step budget in state
How do you make resume safe?	Idempotent pre-interrupt code, idempotency keys on writes, deterministic interrupt order
Where would you not use LangGraph?	A straight pipeline with no branching, state or pause; plain code is clearer

Chapter 8: LangSmith fundamentals

Source: LangSmith documentation (observability and evaluation concepts), fetched 2026-10-12. Your resume lists LangGraph and LangChain but not LangSmith, and your own evaluation framework was a custom harness. Know LangSmith as the standard tool and say how it would map; do not claim you ran it unless you did.

8.1 Observability model

Term	Meaning
Run	One unit of work: a model call, a prompt-formatting step, a retrieval. Like an OpenTelemetry span
Trace	All runs for one operation, linked by a trace id (capped at 25,000 runs)
Thread	A multi-turn session: a sequence of traces, grouped with a `thread_id` metadata key
Trajectory	The flat ordered list of human, AI and tool messages in a thread, with nesting removed
Project	The container for one application's traces (agent-based workspaces organise by agent and environment instead)
Feedback	A score attached to a run (continuous or categorical): from users, code or an LLM judge
Tags and metadata	Strings and key-value pairs for filtering and grouping (app version, environment, tenant)

Getting traces in: framework integrations trace automatically (LangChain, LangGraph, OpenAI, Anthropic and others); manual instrumentation uses `@traceable`, the `trace` context manager or the `RunTree` API. Retention on SaaS is 180 days; datasets persist indefinitely, so put anything you need to keep into a dataset. Monitoring: dashboards for quality and performance over time, and alerts. Deployment options: cloud, hybrid, self-hosted. *(Environment variable names such as `LANGSMITH_TRACING` were not on the fetched page; they are standard, but verify.)*

8.2 Evaluation model

Term	Meaning
Dataset	A collection of examples
Example	An input and a reference output (plus metadata). Reference outputs go to evaluators only, never to your app
Experiment	One version of your app run over a dataset: outputs, scores and traces, comparable side by side
Evaluator	Returns feedback `{key, score or value, comment}`. Kinds: **code** (deterministic checks), **LLM-as-judge** (reference-free or reference-based), **human** (annotation queues, single-run or pairwise), **pairwise** (compare two versions)
Offline evaluation	On a dataset with reference outputs: benchmarking, regression tests, backtesting before release
Online evaluation	On production runs with no reference outputs: monitoring, anomaly detection, collecting failures to turn into new examples

"Testing asserts correctness; evaluation measures performance against metrics." Offline metrics can become regression tests (a new version must beat a baseline on key metrics), written with pytest or Vitest/Jest.

8.3 How it maps to your eval story

Your system	LangSmith equivalent
524-utterance suite, 499-row daily regression	A dataset plus experiments per release
PASS / FAIL / SOFT_FAIL harness results	Evaluator feedback with a score and a comment
LLM judge, κ 0.649	An LLM-as-judge evaluator; track agreement against human labels and calibrate it
Replaying production failures	Online runs flagged by feedback, added to the dataset as new examples
No CI gate	A regression test that fails the build if the per-category pass rate drops below a floor
Planner trace steps (LLM, Function, Transition, OutputEvaluation)	Runs inside a trace; grounded/ungrounded as feedback on the run

How to say it: "*Our harness was custom, but the structure is the standard one: a dataset of utterances with post-condition checks, evaluators at each seam, experiments per release, online failures fed back into the dataset. The gap was the last step, a release gate.*"

8.4 Questions

Question	Answer
Trace versus run?	A run is one step; a trace is the tree of runs for one request
How do you debug a bad agent turn?	Open the trace, find the first node whose output diverged, check retrieval results and tool inputs
How do you evaluate a multi-turn agent?	Thread-level evaluation over the trajectory, plus per-turn checks on tool choice and arguments
How do you avoid judging with the same model that generated?	Different judge model, calibrated against human labels, with agreement tracked
What do you put in metadata?	App and graph version, tenant, model, prompt version, so a regression can be tied to a release
Privacy?	Tracing sends prompts and tool I/O to the tracing backend. Redact PII, choose self-hosted for regulated data, and respect retention

Chapter 9: Mock hiring-manager interview

Cover the answers. Say yours aloud, then compare. Keep each answer to 60–90 seconds.

Q1. Walk me through the flow of a production AI agent. Use 5.1: six layers, then what makes it production-grade (deterministic authorization, approval before writes, per-hop timeouts, attributable steps, fallback graph, staged rollout), then name two real weaknesses.

Q2. Where have you contributed? Give the pitch from the top of this document: planner and reasoner V1 → V2, MCP server and Headless 360 onboarding with selective tool calling, retrieval and grounding through connectors, and the access-management workflows, Slack agent and evals on your resume. End on the principle: the model proposes, deterministic code authorises.

Q3. What did V1 look like and why did you rebuild it? Topic-and-instruction ReAct loop, about 45 bespoke Java actions, no clarifying questions, an upfront plan that took 8–12 seconds. Rebuilt as a supervisor-to-specialist graph so that order is structural, nodes are testable and the pause is durable. Cost: some latency (HLD reported 10–20% slower).

Q4. Explain the supervisor and specialists. Supervisor routes on state and gets control back after each specialist. Knowledge retrieves, Reasoner plans, Executor runs with no LLM, Discovery looks up org records. Say it is a logical role, split across the session worker and graph state in the implementation.

Q5. Why is the Executor LLM-free? What runs must equal what the user approved. A re-reasoning executor breaks that guarantee and makes behaviour unrepeatable.

Q6. Explain this plan. *(your JSON)* Stage 1 serial: resolve "Ravi" to `userId`. Stage 2 parallel: permission sets, public groups and queues, each depending only on that id; stage latency equals the slowest step. `$ref` is resolved by code. Reads need no approval; any write stage is serial and behind the approval gate. Then volunteer: parallel stages are the HLD's proposal; the verified V2 runs a todo list.

Q7. What if there are three users called Ravi? `FindUser` returns several matches; `$ref ...userId` assumes one. A cardinality check routes to a clarification turn before stage 2.

Q8. How is the plan validated? Schema; tool on the enabled list; every `$ref` points to an earlier stage; inputs match the tool schema; no write in a parallel stage. Be honest that runtime plan validation beyond approval was not found, and that this is what you would add.

Q9. What if one parallel step fails? Reads: continue with partial results and say which lookup failed. Writes: fail fast and report exactly what ran. No compensation today; the fix is idempotent create-or-get keyed by a request id, per-step outcome records, and resume or undo.

Q10. Why no retries on actions? Writes are not idempotent, so a blind retry can create a duplicate. Idempotency keys first, retries second. The LLM Gateway retries up to 10 times because model calls have no side effects.

Q11. Where does state live, and what happens on a restart mid-approval? Graph state in Redis next to the reasoner. TTL and failover behaviour not found; state the requirement: TTL longer than the longest plausible approval wait, a defined outcome for an in-flight approval, and an explicit expiry so a stale plan cannot be approved against a changed org.

Q12. How does `interrupt()` work and what goes wrong? It pauses at a checkpoint and returns when you resume with `Command(resume=...)` on the same thread id. The node re-runs from its first line on resume, so pre-interrupt code must be idempotent. Do not wrap it in a bare `try/except`; keep order deterministic; no `while True` around it.

Q13. What is Headless 360 and what did you do? Capability exposed as API, tool discovery and MCP execution so any surface calls one path with one set of checks. Describe the onboarding checklist in 3.4. Give the eight only if you can name them.

Q14. Whose identity is on the MCP call? The requester, through the authenticated session. Know the failure mode: an integration identity would make Core check the wrong user's permissions (confused deputy). Test: call as a low-privilege user and expect a 403.

Q15. What is selective tool calling? Two layers. The MCP server only advertises tools the caller is entitled to. Inside the agent, retrieval narrows the menu: documents carry a `tools:` list, tool retrieval returns the top 5 above 0.7, and the Executor only runs enabled tools.

Q16. Walk me through retrieval and grounding. Chapter 4.1: parallel branches, hybrid search, rerank (320 chunks, 12 s), Core permission filter, LLM document selection, then the answer is checked for citations; ungrounded output is flagged. Weakest link: top-5 recall and silent degradation.

Q17. DLO versus DMO. DLO is the raw landed copy in the source's shape; DMO is the harmonised canonical object mapped from it; chunking, embeddings and a hybrid index sit on top, with a named retriever. The HLD adds metadata DLOs (mDLO) with an extraction status machine; the code shows manual ingestion and a plain DLO from an S3 data stream.

Q18. How fresh is the knowledge and how is it isolated? Freshness is bounded by manual refresh plus a jar bump of 12–24 hours. One shared tenant holds no customer data, so isolation is about authenticating the calling org through token claims. Customer-data retrievers are isolated per org and data space.

Q19. How does a change reach production? How do you roll back? Chapter 5.3, by deployable. Graph promotion every Tuesday; per-org switch to `Stable` reverts the graph only, not actions, templates or NLDs. Services use Argo canary or blue/gold. Then name what is missing: no eval gate on promotion.

Q20. How do you know it is healthy? Per-layer logs, planner trace steps, grounding outcome. Alerts exist for the retriever, planner and federator; none found for the gateway, reasoner or Setup agent; no SLOs. You would define SLIs: turn success, approval rate, time to approve, p95 per hop, and page on burn rate.

Q21. If the agent were compromised, what is the worst it can do? Bounded by the user's own permissions, the action allowlist, the surface blocklist and the approval layers. Be honest: the bound is weaker than intended because one generic write tool lacks confirmation and single-task plans auto-approve. Fix: one deterministic confirmation gate for all writes.

Q22. How do you test a non-deterministic planner? Test at the seams: retrieval recall@k, tool-selection accuracy, plan validity, a pure-code Executor, end-to-end golden tasks, and replayed production failures. Numbers: 524 utterances, 81.7% pass, 88.3% stable, judge κ 0.649, no CI gate. Fix: a per-category floor on promotion.

Q23. What is your biggest risk and what do you fix first? Non-uniform approval over a partly non-idempotent write path. First: uniform approval plus idempotency; second: an eval release gate; third: SLOs and alerts, because you cannot improve what you cannot see.

Q24. How do you use LangSmith? Say what is true. The structure maps one to one (chapter 8.3): datasets, evaluators, experiments, online failures fed back, and a regression gate.

Q25. Why should we believe those numbers? Use the defence line from chapter 6, one per number, with method and caveat. If you cannot state the method, drop the number and say the mechanism.

Q26. What would you do differently? Idempotency keys and compensation; one end-to-end deadline propagated downward with stream heartbeats; scheduled, versioned knowledge ingestion with an eval gate before an index swap; versioning graph, actions and NLDs as one release unit.

Chapter 10: Study order and cheat sheet

10.0 If you only have 3 hours

Time	Do	Where
0:00 to 0:20	Ownership wording for each pillar; memorise the "shipped vs design" table	1.2, 3.6.0
0:20 to 1:10	MCP and selective tool calling: read 3.6, retype `select_tools` and `confirm_action`, say the 9 challenges out loud	3.2 to 3.6
1:10 to 1:50	Planner and reasoner: supervisor, approval with `interrupt()`, executor, `validate_plan`	Ch. 2
1:50 to 2:15	Infrastructure answers (the axis that cost the last round)	5.3
2:15 to 2:30	Retrieval chain and "not found" list; resume numbers	Ch. 4, 6
2:30 to 3:00	Mock interview out loud, once, recorded; fix weak answers	Ch. 9

10.1 Order (highest value first)

1. **Settle ownership wording (30 minutes).** Chapter 1.2. Decide your exact first sentence for each pillar, and rehearse the answer to "what exactly did you write?" until it is one breath and specific.
2. **Reproduce chapter 2 code from a blank page.** State with reducers, supervisor router, approval with `interrupt()`, executor with `Send`, `resolve_refs`, `validate_plan`. If you can write these, you can defend the reasoner.
3. **Infrastructure for what you claim (chapter 5.3).** Container, gunicorn drain, HPA, mesh, Argo canary, blue/gold, feature gates. This is the axis that cost the last round.
4. **MCP and identity (chapter 3).** Handshake, `tools/list` versus `tools/call`, identity propagation and the confused-deputy test.
5. **Retrieval, DLO and DMO (chapter 4).** The chain, pre- versus post-filter, isolation, freshness, and what is not found.
6. **Numbers and the resume (chapter 6).** One defence line per number.
7. **LangGraph and LangSmith fundamentals (chapters 7–8)** for the below-the-architecture questions.
8. **Mock interview (chapter 9) out loud, twice.**

10.2 Say / do not say

Say	Do not say
"The model proposes; deterministic code authorises."	"The agent decides what to run."
"The design describes a supervisor; in the implementation the role is split."	"There is a ConversationManager service."
"Parallel stages are the proposed schema."	"Parallel execution is live." *(unless you confirm it)*

Say	Do not say
"I didn't find production chunk size or embedding model."	A guessed value
"There is no calibrated confidence gate."	"Plans are confidence-scored before execution."
"Ingestion is manual today."	"Ingestion runs on a schedule."
"Rollback to Stable reverts the graph only."	"We can roll back everything."

10.3 Numbers cheat sheet

Number	Meaning
45	Bespoke actions in the V1 pilot (HLD)
Top 5 / 0.7	Tool retrieval candidates / similarity threshold
10 (max 2048)	Retriever default topK
320 / 12 s	Reranker chunks / timeout
35 s	Gateway retriever query timeout
about 11 s / 60 s	Search deadline / query service deadline
60 s (90 s in one region)	Action timeout, reasoner → planner → Core
connect 5 s / read 90 s	Reasoner → LLM Gateway
25 s (Bedrock 90 s)	LLM Gateway per-provider timeout
300 s / 601 s	Mesh timeout / stream cap
10 retries	LLM Gateway; none on actions
200 / org / min	Rate limit
about 5 and 12	LLM calls per request at P95 and P99
81.7% / 88.3% / κ 0.649	Eval pass / stable / judge agreement
524 / 499	Utterance suite / daily regression rows
Tuesday	Weekly graph promotion
12–24 h	Knowledge corpus change reaching Core
50	Max membership assignments per call
3–9 / 3–12	HPA: LLM Gateway / AI Gateway
about 21	Reasoner rollout instances
p50 about 35 s, p95 about 98 s	MCP proxy local sample (directional)
10–20%	V2 slower than the V1 planner (HLD, Jan 2026, not re-verified)

10.4 What I did not read

The 843 KB GitLab master prep PDF was checked by table of contents and keyword search, not read page by page; its Salesforce chapters (15.x) duplicate material already in the A4S guide and drills. The LangGraph subgraph, streaming and `get_state` pages, and LangSmith dashboards, alerts and environment-variable pages, were not in the fetched content, so those items are marked as general knowledge to verify.